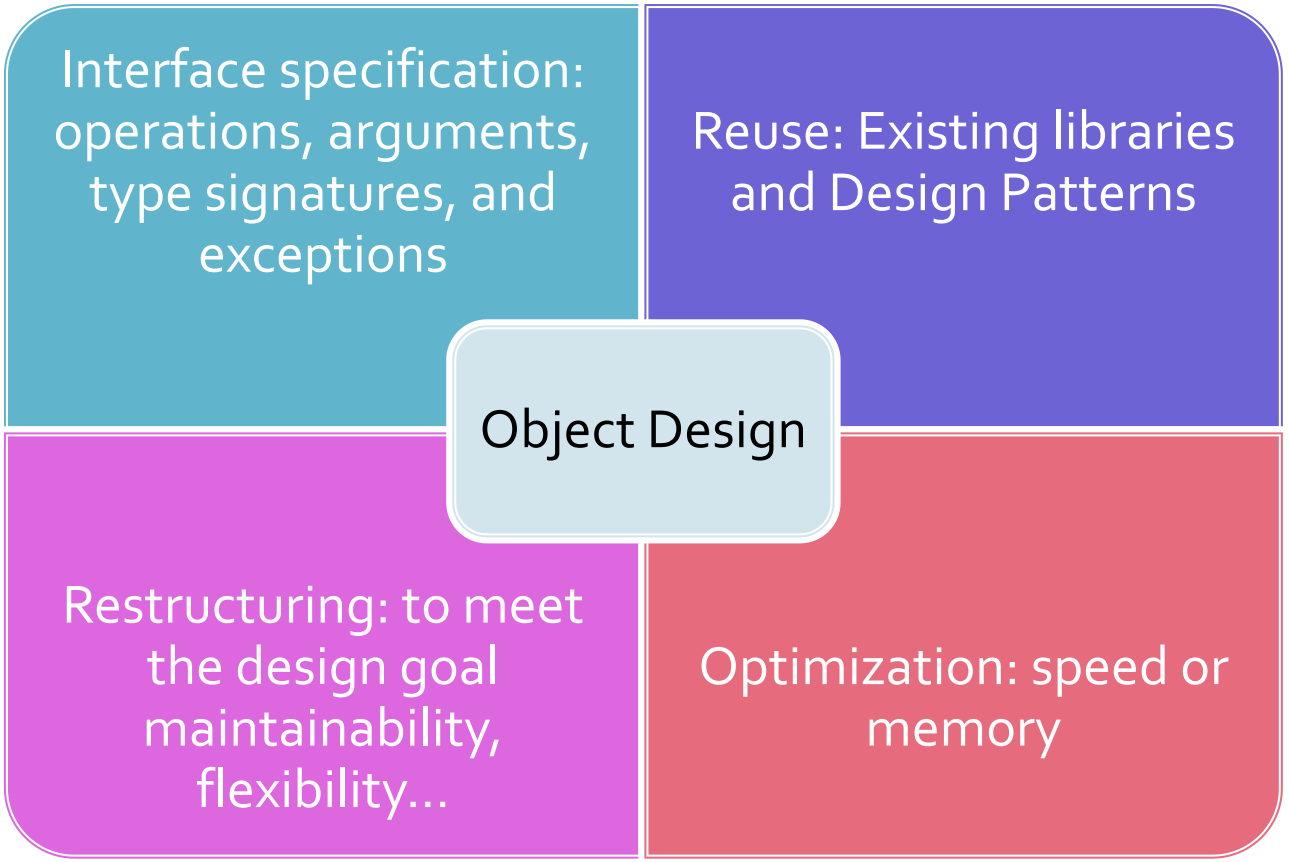
CZ2006/CE2006 Software Engineering

Lecture 14: Design Patterns

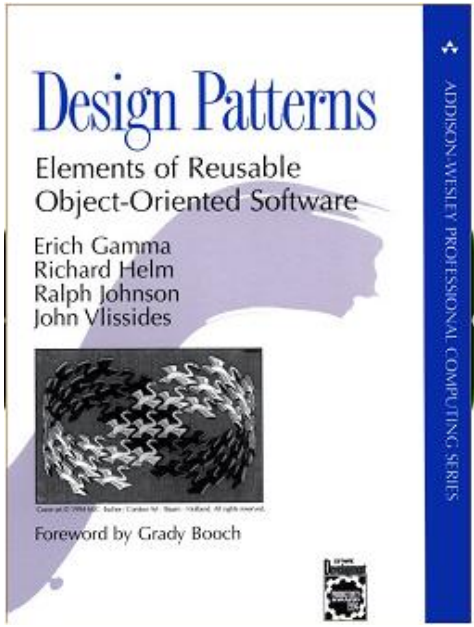
Liu Yang

Recap of Object Design



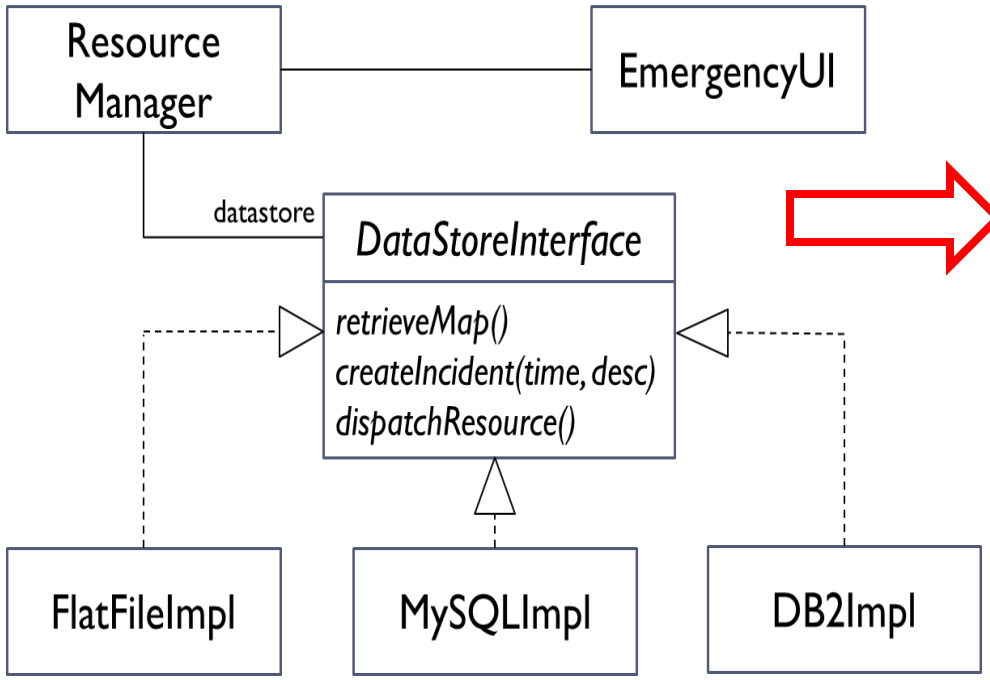
GoF Patterns (23)

- *Creational Patterns*
  - Abstract Factory
  - Builder
  - **Factory Method**
  - Prototype
  - Singleton
- *Structural Patterns*
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - **Façade**
  - Flyweight
  - Proxy
- *Behavioral Patterns*
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - **Observer**
  - State
  - **Strategy**
  - Template Method
  - Visitor



What To Do When System Starts Up

- Create objects, and associate them up according to the class diagram




```
// create data store object user chooses
DataStoreInterface datastore = null;
if(datastoreOption.equals("M")) {
    datastore = new MySQLImpl();
} else if(datastoreOption.equals("D")) {
    datastore = new DB2Impl();
} else if(datastoreOption.equals("F")) {
    datastore = new FlatFileImpl();
} else {
    datastore = new FlatFileImpl();
}
```

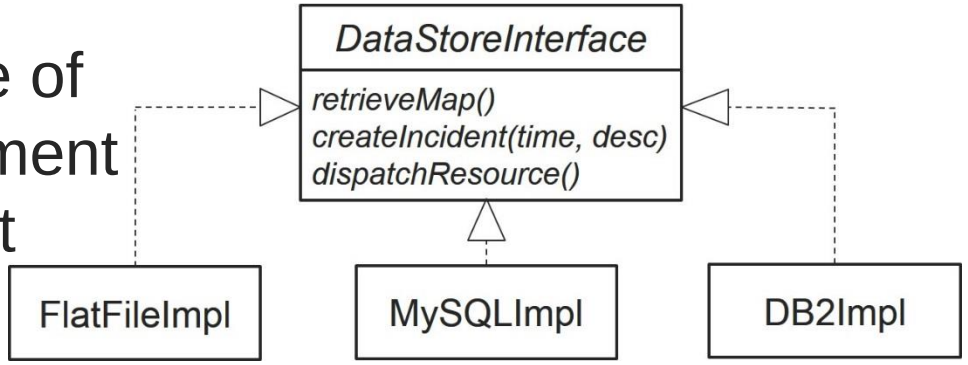
```
// create resource manager object
ResourceManager resourceManager =
        new ResourceManager(datastore);

// create emergency ui object
EmergencyUI ui = new EmergencyUI();
ui.setResourceManager(resourceManager);
```

See EMSNoFactory.zip for implementation!

When To Use Factory Pattern?

- When you've a set of classes but don't know exactly which one you'll need to instantiate until runtime.
- When you need to create one of of several classes that implement a common superclass without exposing the creation logic to the client.



- When you want to localize the logic to instantiate objects.

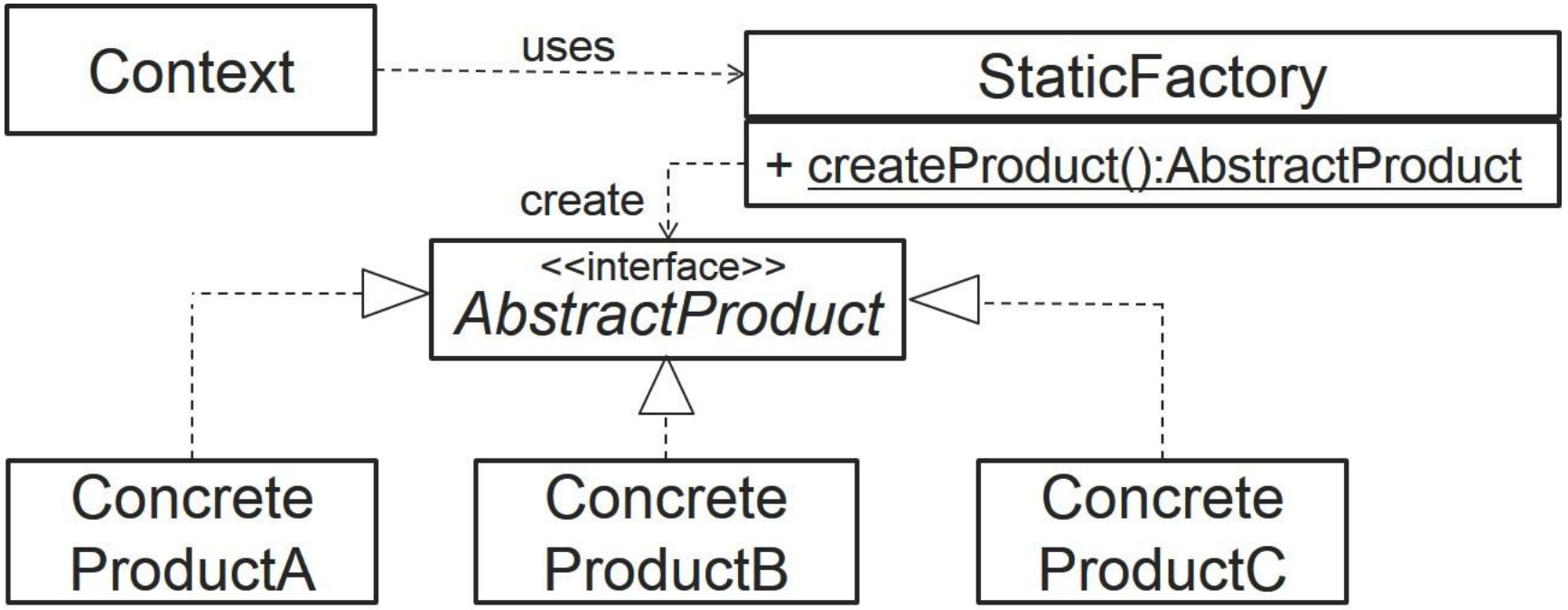
Solution: Factory Pattern

- Defines an interface for creating different objects, without knowing beforehand what sort of objects it needs to create or how the object is created – lets subclasses decide which class to instantiate.

What Are The Design Problems?

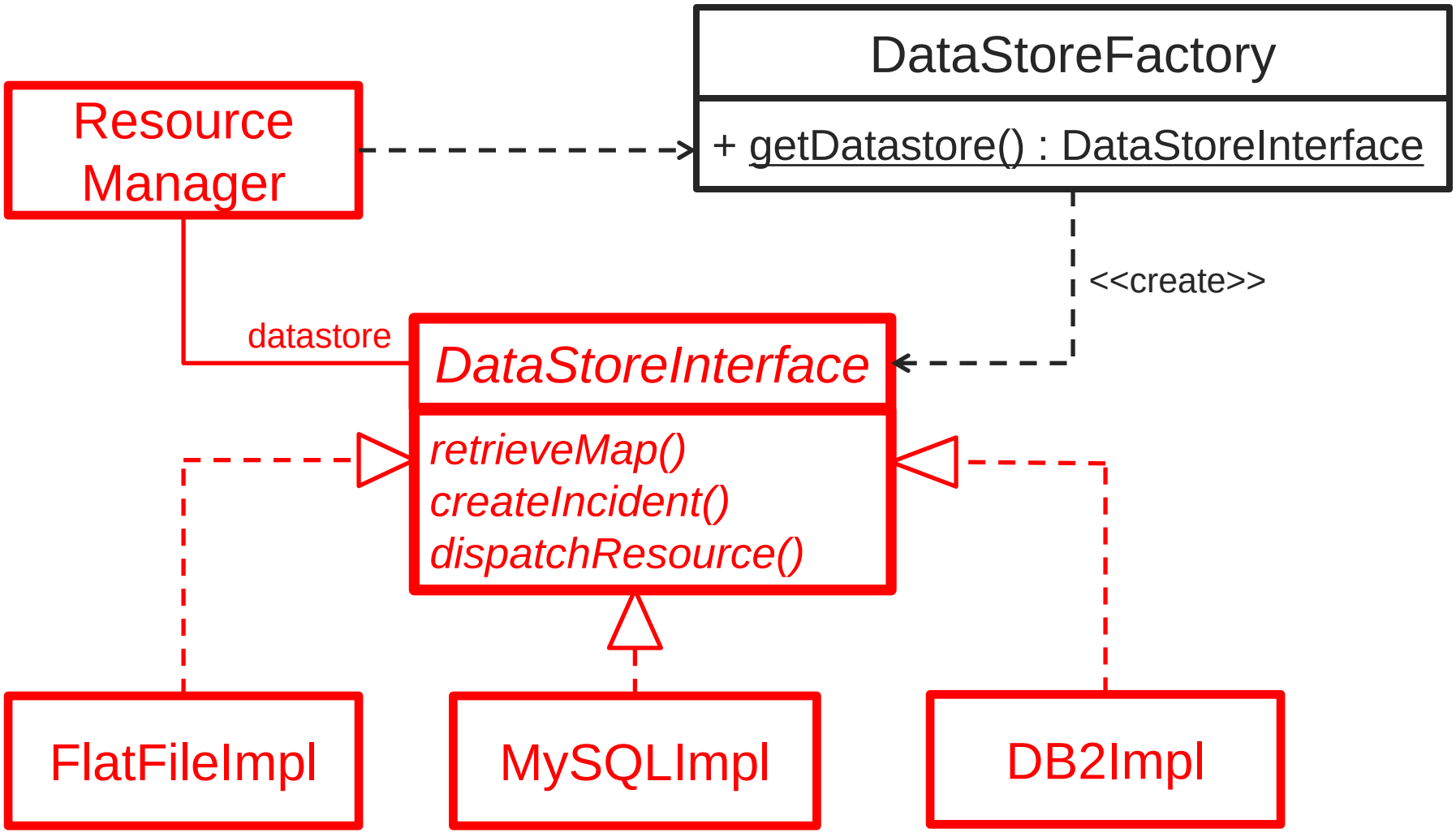
- **Decouple class selection** and **object creation** (abstract instantiation process) from the client where the object is used allowing greater flexibility in object creation
  - Existing subclasses can be replaced, or new subclasses can be added
  - Process of selecting subclass to use and creating object can be complex
  - Client does not know ahead of time which class will be used, and the class can be chosen at runtime

Factory Pattern (a.k.a. Static Factory Pattern) – "Standard" Version (Template)



- **Context** uses **StaticFactory** to obtain an object that implements ***AbstractProduct*** interface.

Data Access: Strategy Design + Factory Design



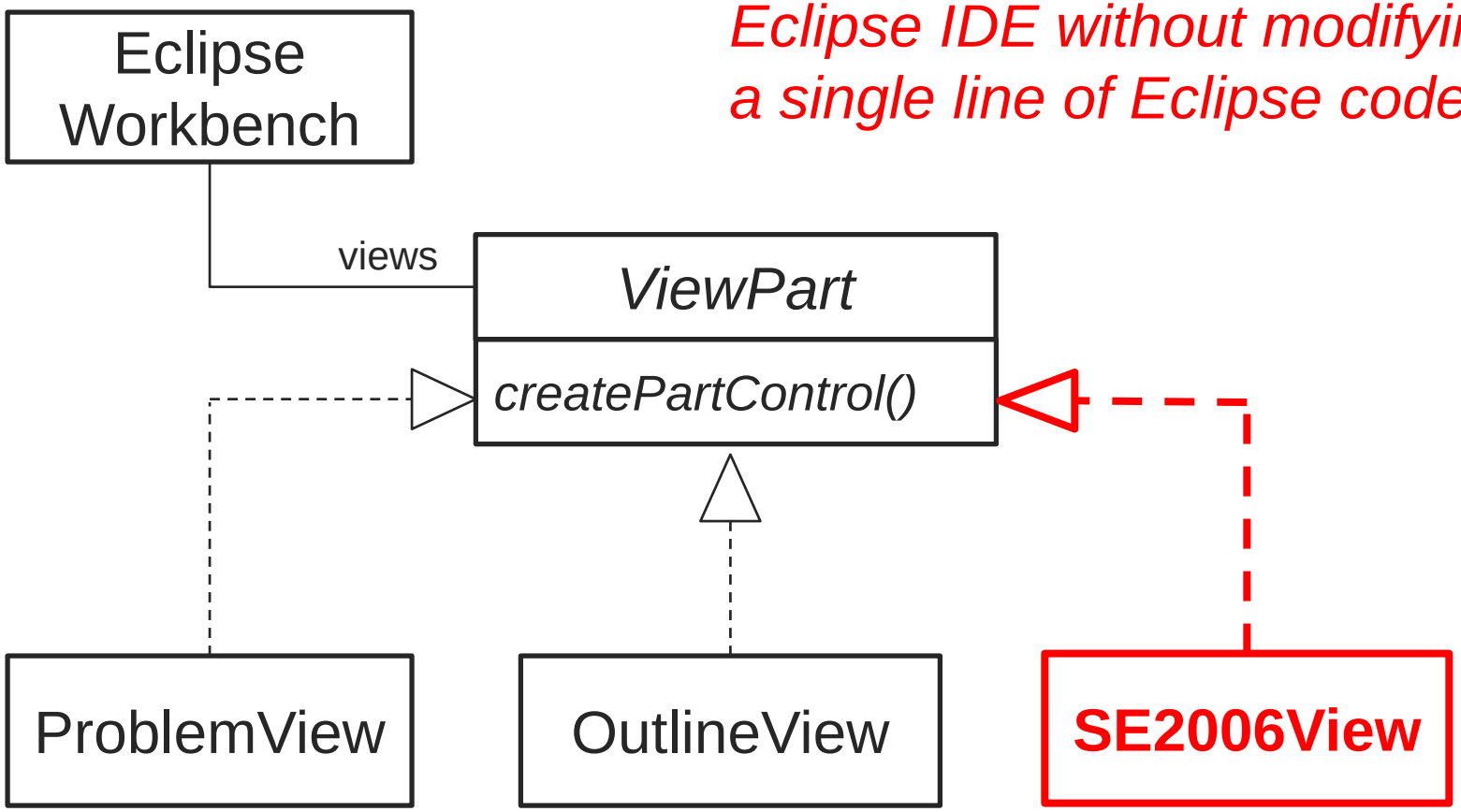
See EMSBasicFactory.zip for implementation

What Are The Benefits?

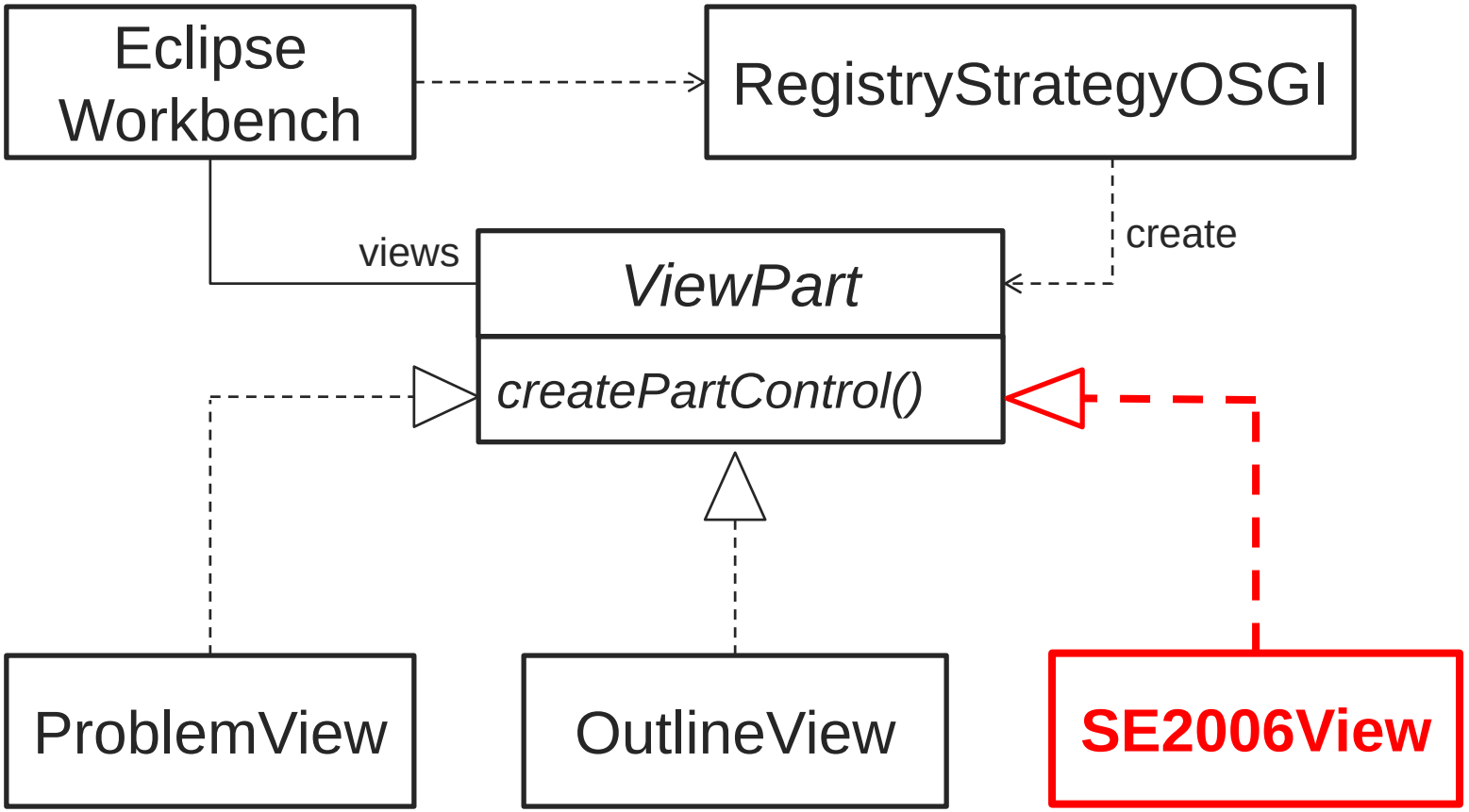
- **Encapsulate object creation**
  - Remove duplicate object creation code from clients (*see EMSNoFactory.zip and EMSBasicFactory.zip*)
  - Centralize class selection and object creation code
- **Easy to extend, replace, or add new subclasses**
  - Replace MySQLImpl1 with MySQLImpl2
  - Support new OracleImpl
- **Easy to change object creation logic**
  - Support Singleton object (*see EMSFactoryUsingSingleton.zip*)
  - Support lazy initialization (*see EMSFactoryLazyInitialization.zip*)

Can You Extend Eclipse IDE?

Can we add SE2006View to Eclipse IDE without modifying a single line of Eclipse code?

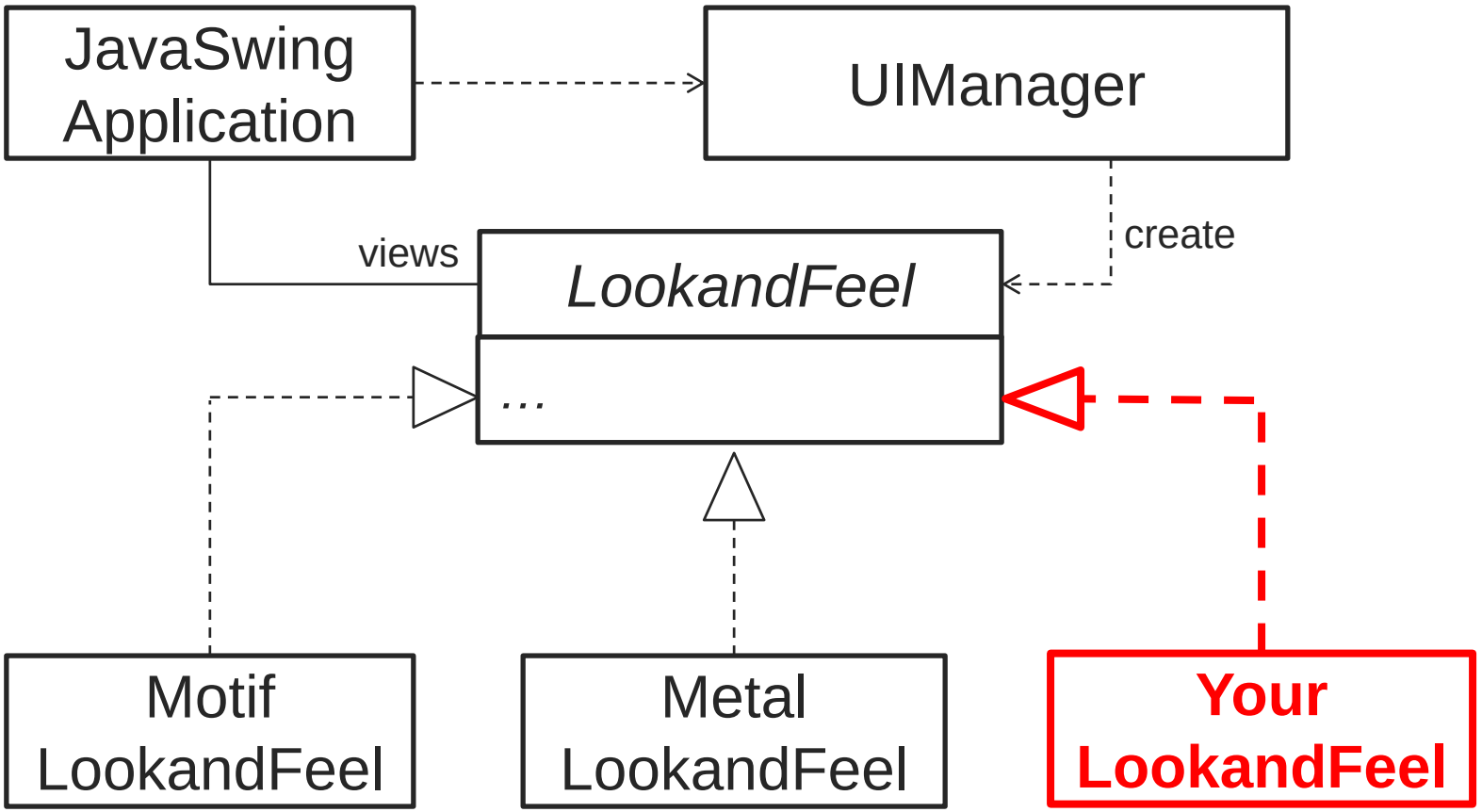


The Magic Power: Strategy Design + Factory Design + Dynamic Loading



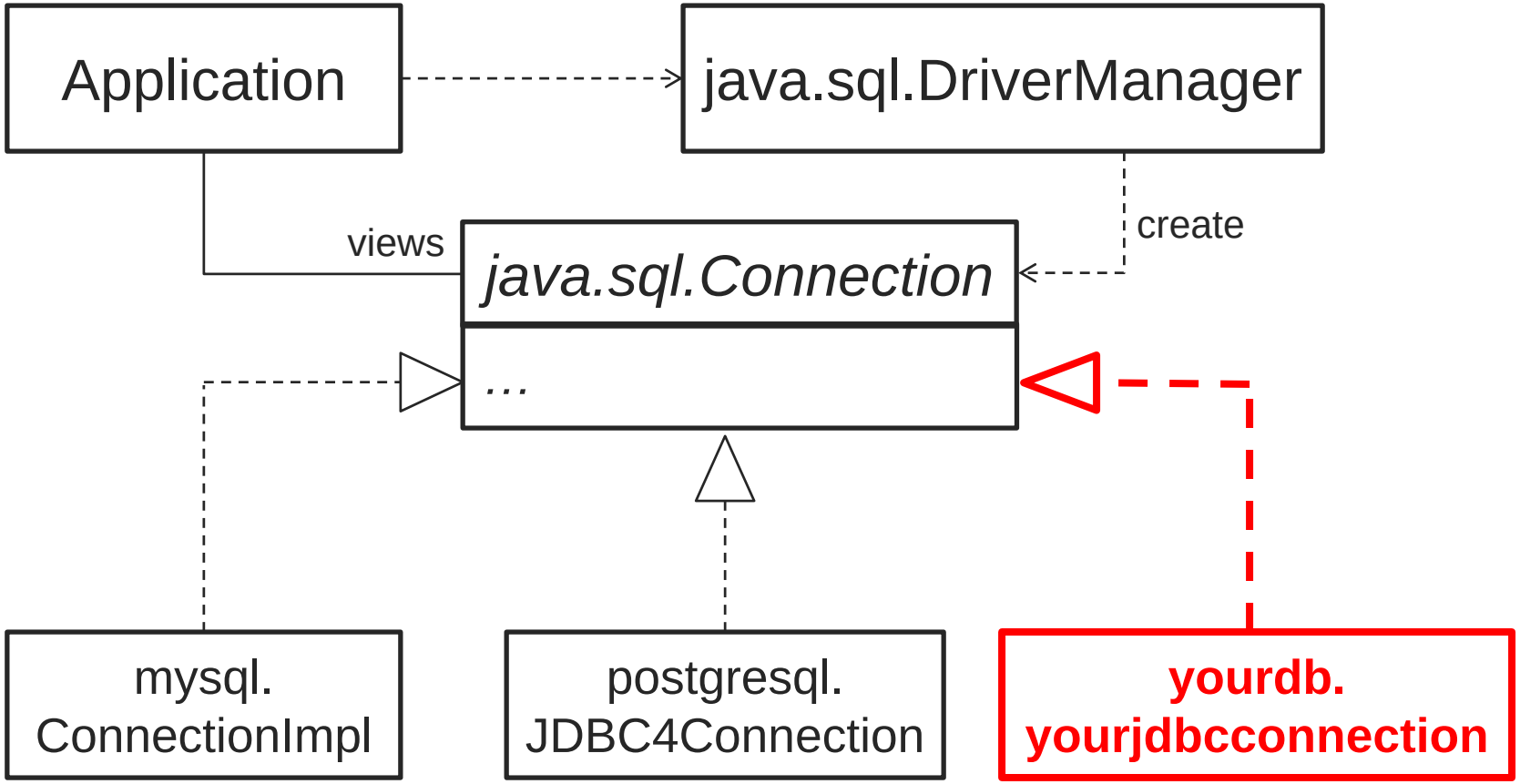
See YouExtendEclipse.zip for code example

Java Swing Framework Look & Feel



Strategy Design + Factory Design + Dynamic Loading

JDBC Database Driver



Strategy Design + Factory Design + Dynamic Loading

Strategy + Factory + Dynamic Loading

- Create any subclass object without specifying the class name in the code
- No need to change a single line of code to use new classes
- Provide the foundation for modern software frameworks to load application-specific classes

See EMSFactoryDynamicLoading.zip for code example

Façade Pattern Definition

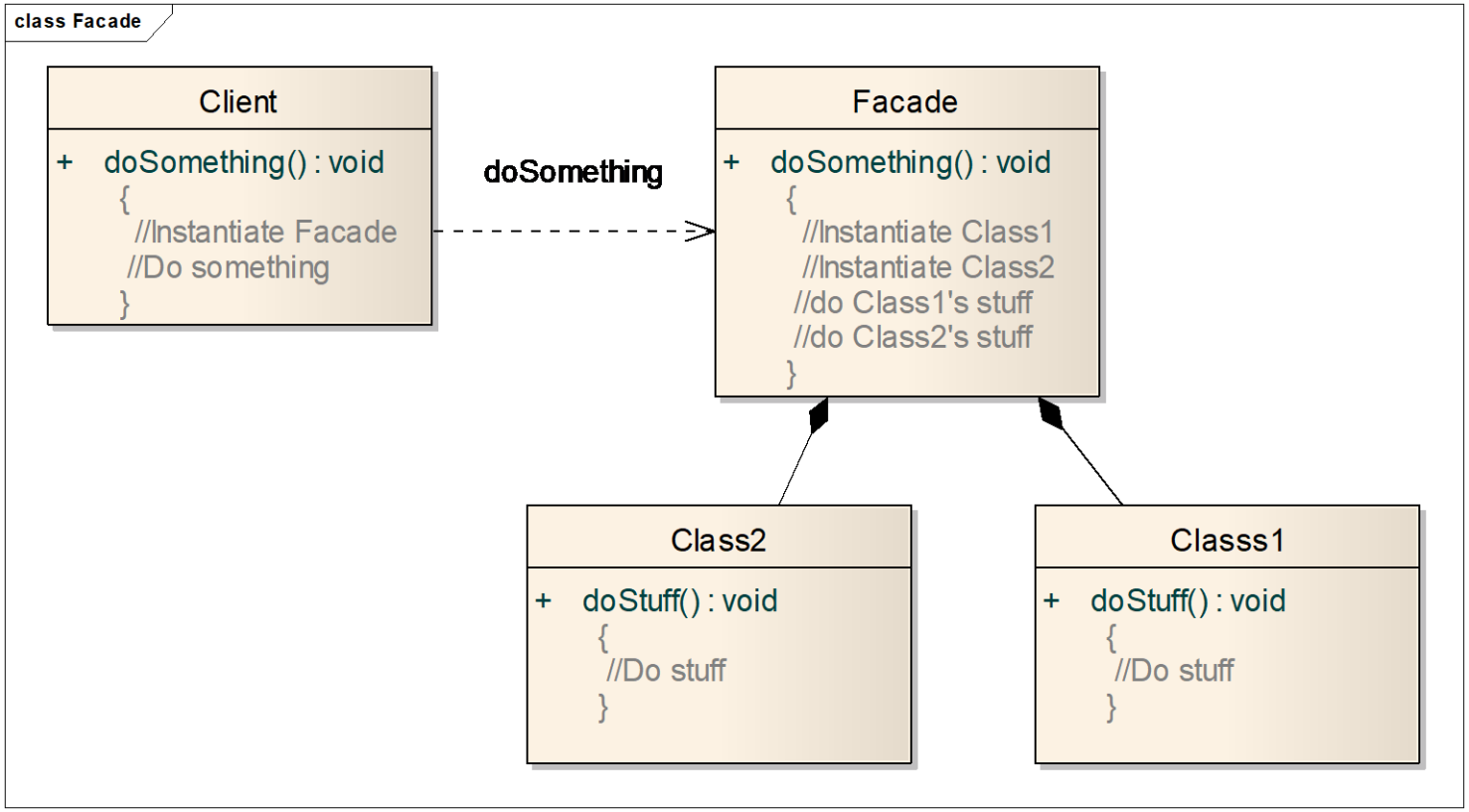
Provides a unified interface to a set of interfaces in a subsystem. Façade defines a higher level interface that makes the subsystem easier to use.

Design Principles

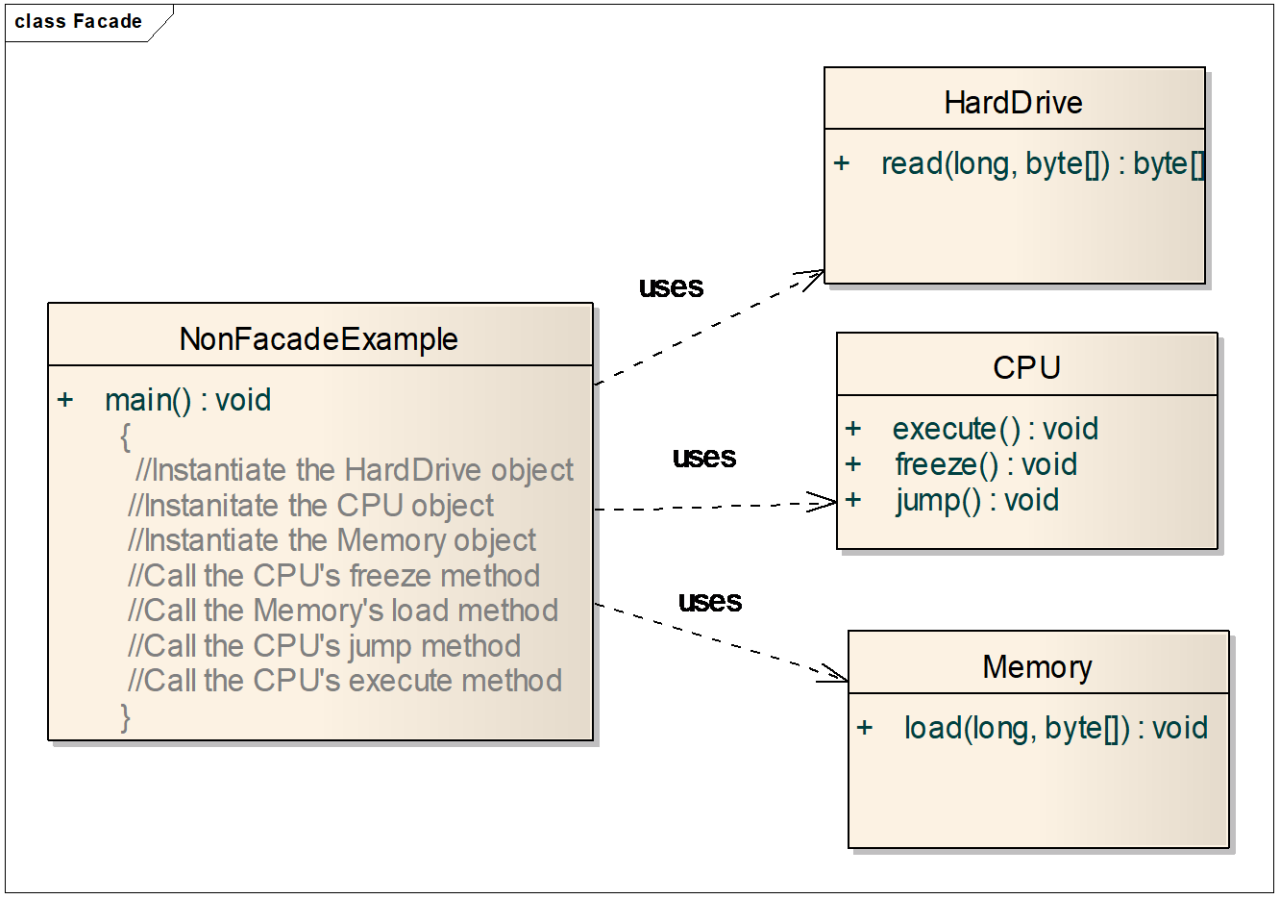
- Identify the aspects of your application that vary and separate them from what stays the same
- Program to an interface, not an implementation
- Favor composition over inheritance
- Strive for loosely coupled designs between objects that interact
- Classes should be open for extension, but closed for modification

- Principle of least knowledge – talk only to your immediate friends

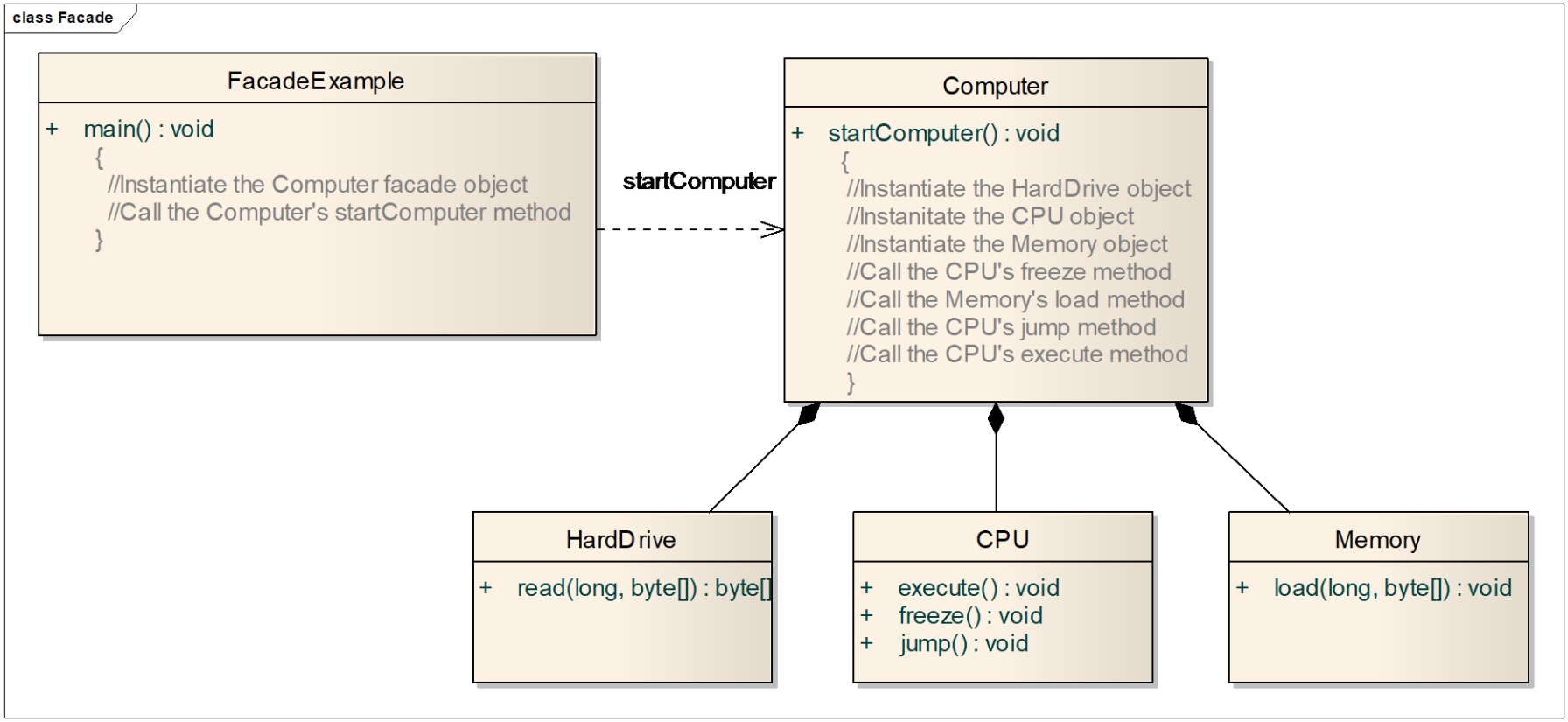
Façade – Class diagram



Façade - Problem



Façade - Solution



Facade

- Pros
  - Makes code easier to use and understand
  - Reduces dependencies on classes
  - Decouples a client from a complex system

- Cons
  - Results in more rework for improperly designed Façade class
  - Increases complexity and decreases runtime performance for large number of Façade classes

Summary of Patterns

- *Creational Patterns*
  - Abstract Factory
  - Builder
  - **Factory Method**
  - Prototype
  - Singleton
- *Structural Patterns*
  - Adapter
  - Bridge
  - Composite
  - Decorator
  - **Façade**
  - Flyweight
  - Proxy
- *Behavioral Patterns*
  - Chain of Responsibility
  - Command
  - Interpreter
  - Iterator
  - Mediator
  - Memento
  - **Observer**
  - State
  - **Strategy**
  - Template Method
  - Visitor